

UNIVERSIDAD
NACIONAL
DE COLOMBIA

ACTIVIDAD 3 DE PROGRAMACION ORIENTADA A OBJETOS

Presentado por: Maria Paula Barrera Avella – 1001220174

Presentado a: Walter Hugo Arboleda Mazo

Asignatura: Programación orientada a objetos

Medellin, Antioquia

Fecha: 22 de octubre

2026

Ejercicio 8.2

```
#Ejercicio 8.2

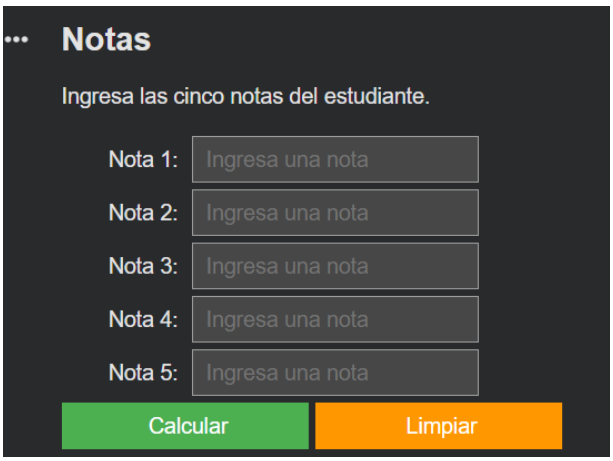
import math
import ipywidgets as widgets
from IPython.display import display, clear_output
# Clase Notas
class Notas:
    def __init__(self, lista_notas):
        self.lista_notas = lista_notas
    def calcular_promedio(self):
        suma = 0
        for nota in self.lista_notas:
            suma += nota
        return suma / len(self.lista_notas)
    def calcular_desviacion(self):
        promedio = self.calcular_promedio()
        suma = 0
        for nota in self.lista_notas:
            suma += (nota - promedio) ** 2
        return math.sqrt(suma / len(self.lista_notas))
    def calcular_menor(self):
        menor = self.lista_notas[0]
        for nota in self.lista_notas:
            if nota < menor:
                menor = nota
        return menor
    def calcular_mayor(self):
        mayor = self.lista_notas[0]
        for nota in self.lista_notas:
            if nota > mayor:
                mayor = nota
        return mayor
# Clase VentanaPrincipal
class VentanaPrincipal:
    def __init__(self):
        self.campos = []
        for i in range(5):
            campo = widgets.Text(
                description=f"Nota {i + 1}:",
                placeholder="Ingresa una nota",
                style={"description_width": "70px"},
                layout=widgets.Layout(width="250px")
            )
            self.campos.append(campo)
        self.boton_calcular = widgets.Button(
            description="Calcular",
            button_style="success",
            icon="calculator"
        )
        self.boton_limpiar = widgets.Button(
```

```
            description="Limpiar",
            button_style="warning",
            icon="trash"
        )
        self.resultados = widgets.Output()
        self.boton_calcular.on_click(
            self.action_performed
        )
        self.boton_limpiar.on_click(
            self.limpiar
        )
    def action_performed(self, boton):
        with self.resultados:
            clear_output(wait=True)
            try:
                lista_notas = []
     # Validar e ingresar las cinco notas
                for campo in self.campos:
                    texto = campo.value.strip().replace(",", ".")
                    if texto == "":
                        raise ValueError(
                            "Debes ingresar las cinco notas."
                        )
                    nota = float(texto)
                    if not math.isfinite(nota):
                        raise ValueError(
                            "Ingresa únicamente números válidos."
                        )
                    if nota < 0 or nota > 5:
                        raise ValueError(
                            "Las notas deben estar entre 0 y 5."
                        )
                    lista_notas.append(nota)
    # Crear el objeto Notas
                notas = Notas(lista_notas)
    # Mostrar los resultados
                print("RESULTADOS")
                print("-" * 35)
                print(
                    f"Promedio: "
                    f"{notas.calcular_promedio():.2f}"
                )
                print(
                    f"Desviación estándar: "
                    f"{notas.calcular_desviacion():.2f}"
                )
                print(
                    f"Nota mayor: "
                    f"{notas.calcular_mayor():.2f}"
                )
                print(
```

```
                f"Nota menor: "
                f"{notas.calcular_menor():.2f}"
            )
        except ValueError as error:
            print(f"Error: {error}")
    def limpiar(self, boton):
        for campo in self.campos:
            campo.value = ""
        with self.resultados:
            clear_output()
    def mostrar(self):
        titulo = widgets.HTML(
            "<h2> Notas</h2>"
            "<p>Ingresa las cinco notas del estudiante.</p>"
        )
        display(titulo)
        display(widgets.VBox(self.campos))
        display(
            widgets.HBox([
                self.boton_calcular,
                self.boton_limpiar
            ])
        )
        display(self.resultados)
# Clase Principal
class Principal:
    @staticmethod
    def main():
        ventana = VentanaPrincipal()
        ventana.mostrar()
# Ejecutar el programa
Principal.main()
```

---

Interfaz de usuario



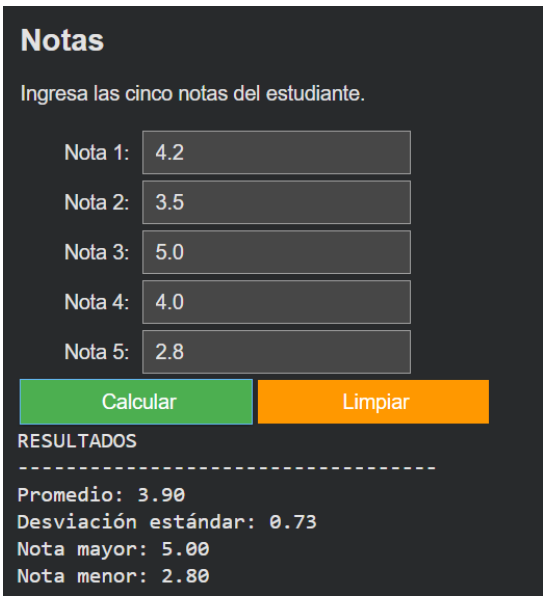



Diagrama de objetos

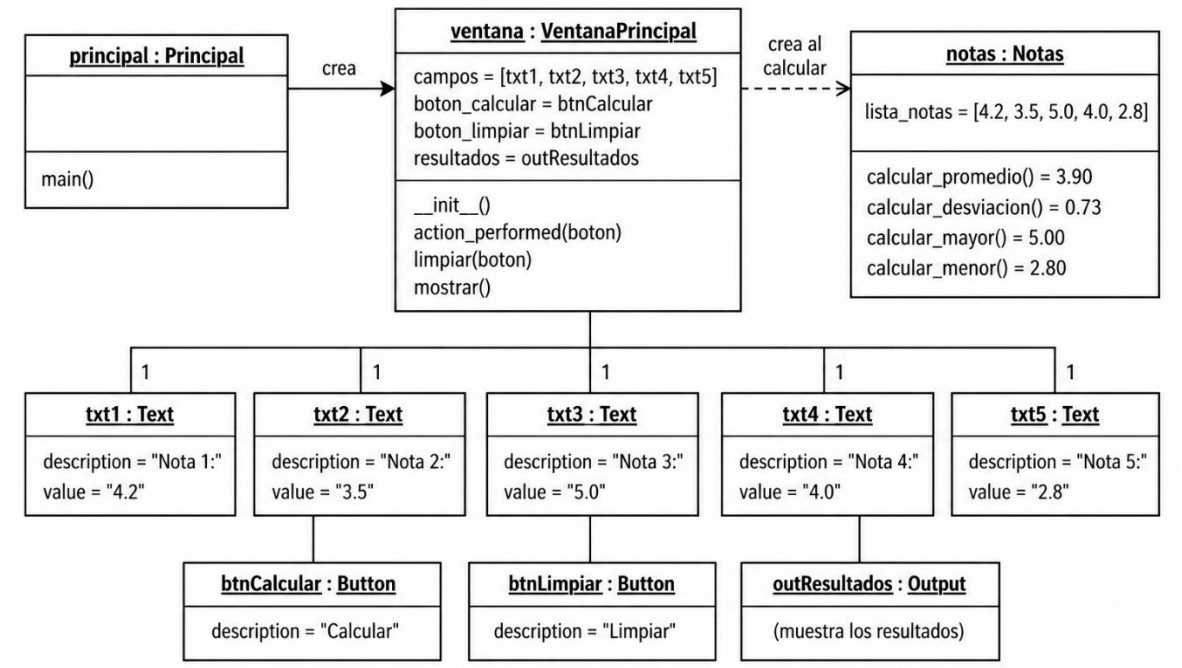
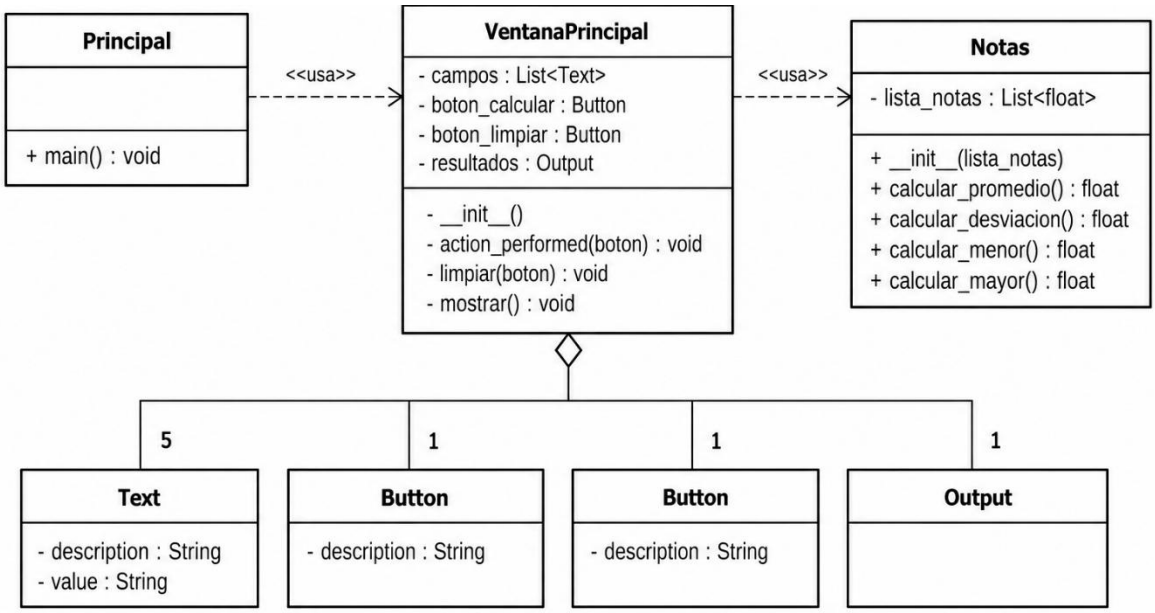
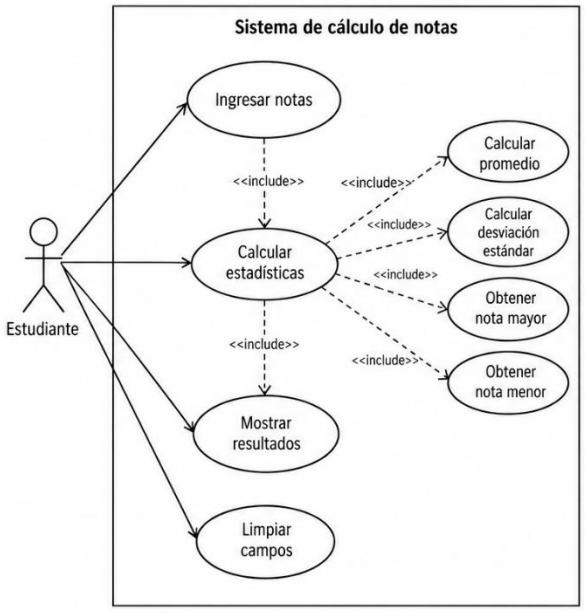



Diagrama de clases



Casos de uso




---

Ejercicio 8.3

```
#Ejercicio 8.3
import math
import ipywidgets as widgets
from IPython.display import display, clear_output
from abc import ABC, abstractmethod
# Clase FiguraGeometrica
class FiguraGeometrica(ABC):
    def __init__(self):
        self._volumen = 0.0
        self._superficie = 0.0
    def set_volumen(self, volumen):
        self._volumen = volumen
    def set_superficie(self, superficie):
        self._superficie = superficie
    def get_volumen(self):
        return self._volumen
    def get_superficie(self):
        return self._superficie
    @abstractmethod
    def calcular_volumen(self):
        pass
    @abstractmethod
    def calcular_superficie(self):
        pass
# Clase Cilindro
class Cilindro(FiguraGeometrica):
    def __init__(self, radio, altura):
```

```
        super().__init__()
        self.radio = self.validar_positivo(radio, "radio")
        self.altura = self.validar_positivo(altura, "altura")
        self.set_volumen(self.calcular_volumen())
        self.set_superficie(self.calcular_superficie())
    @staticmethod
    def validar_positivo(valor, nombre):
        valor = float(valor)
        if not math.isfinite(valor) or valor <= 0:
            raise ValueError(
                f"El valor de {nombre} debe ser mayor que cero."
            )
        return valor
    def calcular_volumen(self):
        volumen = math.pi * self.altura * self.radio ** 2
        return volumen
    def calcular_superficie(self):
        area_lateral = 2 * math.pi * self.radio * self.altura
        area_bases = 2 * math.pi * self.radio ** 2
        return area_lateral + area_bases
# Clase Esfera
class Esfera(FiguraGeometrica):

    def __init__(self, radio):
        super().__init__()
        self.radio = Cilindro.validar_positivo(
            radio, "radio"
        )
        self.set_volumen(self.calcular_volumen())
        self.set_superficie(self.calcular_superficie())
    def calcular_volumen(self):
        volumen = (4.0 / 3.0) * math.pi * self.radio ** 3
        return volumen
    def calcular_superficie(self):
        superficie = 4 * math.pi * self.radio ** 2
        return superficie
# Clase Piramide
class Piramide(FiguraGeometrica):
    def __init__(self, base, altura, apotema):
        super().__init__()
        self.base = Cilindro.validar_positivo(
            base, "base"
        )
        self.altura = Cilindro.validar_positivo(
            altura, "altura"
        )
        self.apotema = Cilindro.validar_positivo(
            apotema, "apotema"
        )
        self.set_volumen(self.calcular_volumen())
        self.set_superficie(self.calcular_superficie())
```

```
    def calcular_volumen(self):
        volumen = (self.base ** 2 * self.altura) / 3.0
        return volumen
    def calcular_superficie(self):
        area_base = self.base ** 2
        area_lateral = 2 * self.base * self.apotema
        return area_base + area_lateral
# Clase VentanaFigura
class VentanaFigura:
    def __init__(self, tipo, campos):
        self.tipo = tipo
        self.campos = campos
    def obtener_valores(self):
        valores = {}
        for nombre, campo in self.campos.items():
            texto = campo.value.strip().replace(",", ".")
            if texto == "":
                raise ValueError(
                    f"Debes ingresar el valor de {nombre}."
                )
            valor = float(texto)

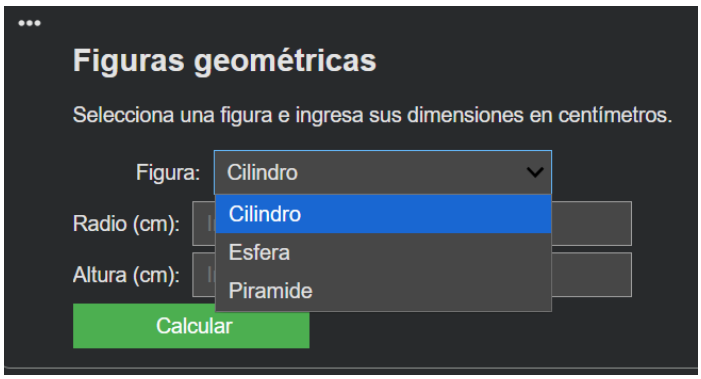
            if not math.isfinite(valor):
                raise ValueError(
                    f"El valor de {nombre} no es válido."
                )
            valores[nombre] = valor
        return valores
# Clase VentanaPrincipal
class VentanaPrincipal:
    def __init__(self):
        self.tipo = widgets.Dropdown(
            options=["Cilindro", "Esfera", "Piramide"],
            value="Cilindro",
            description="Figura:"
        )
        self.formulario = widgets.VBox()
        self.campos = {}
        self.boton_calcular = widgets.Button(
            description="Calcular",
            button_style="success",
            icon="calculator"
        )
        self.resultados = widgets.Output()
        self.tipo.observe(
            self.actualizar_campos,
            names="value"
        )
        self.boton_calcular.on_click(
            self.action_performed
        )
```

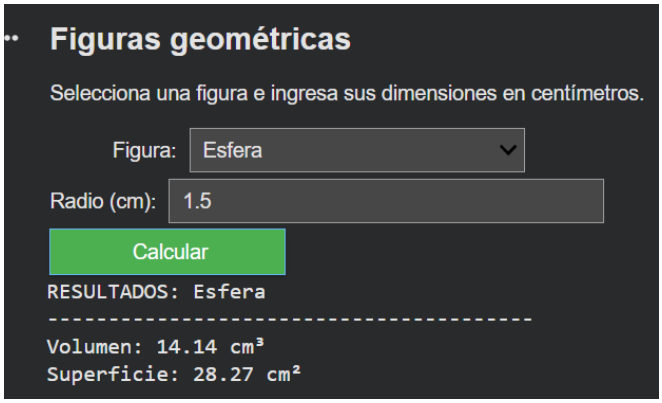
```
    self.actualizar_campos()
def actualizar_campos(self, cambio=None):
    definiciones = {
        "Cilindro": [
            ("radio", "Radio (cm):"),
            ("altura", "Altura (cm):")
        ],
        "Esfera": [
            ("radio", "Radio (cm):")
        ],
        "Piramide": [
            ("base", "Base cuadrada (lado en cm):"),
            ("altura", "Altura (cm):"),
            ("apotema", "Apotema (cm):")
        ]
    }
    self.campos = {}
    for nombre, etiqueta in definiciones[self.tipo.value]:
        self.campos[nombre] = widgets.Text(
            description=etiqueta,
            placeholder="Ingresa un valor",
            style={"description_width": "initial"},
            layout=widgets.Layout(width="350px")
        )
    self.formulario.children = tuple(
        self.campos.values()
    )
    with self.resultados:
        clear_output()
def action_performed(self, boton):
    with self.resultados:
        clear_output(wait=True)
        try:
            ventana = VentanaFigura(
                self.tipo.value,
                self.campos
            )
            valores = ventana.obtener_valores()
            clases = {
                "Cilindro": Cilindro,
                "Esfera": Esfera,
                "Piramide": Piramide
            }
            figura = clases[self.tipo.value](**valores)
            print(f"RESULTADOS: {self.tipo.value}")
            print("-" * 40)
            print(
                f"Volumen: {figura.get_volumen():.2f} cm³"
            )
            print(
                f"Superficie: "
```

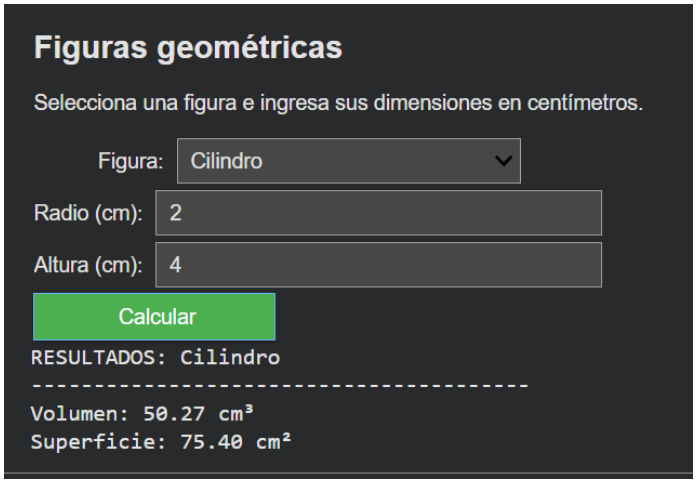
```
                f"{figura.get_superficie():.2f} cm²"
            )
        except (ValueError, TypeError) as error:
            print(f"Error: {error}")
    def mostrar(self):
        display(widgets.HTML(
            "<h2> Figuras geométricas</h2>"
            "<p>Selecciona una figura e ingresa sus dimensiones "
            "en centímetros.</p>"
        ))
        display(self.tipo)
        display(self.formulario)
        display(self.boton_calcular)
        display(self.resultados)
# Clase Principal
class Principal:
    @staticmethod
    def main():
        ventana = VentanaPrincipal()
        ventana.mostrar()
# Ejecutar el programa
Principal.main()
```

---

Interfaz de usuario



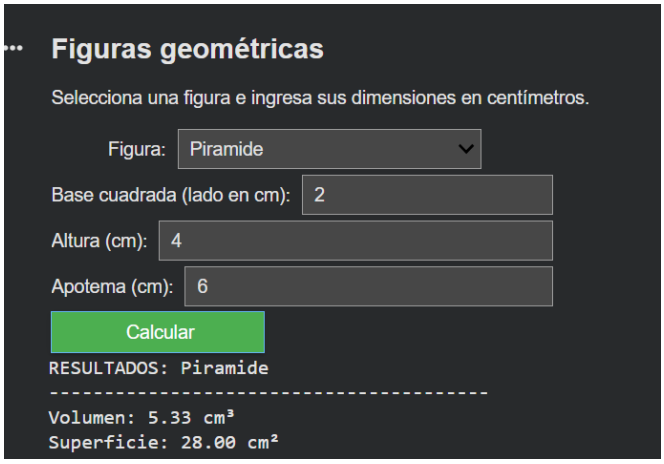



Diagrama de objetos

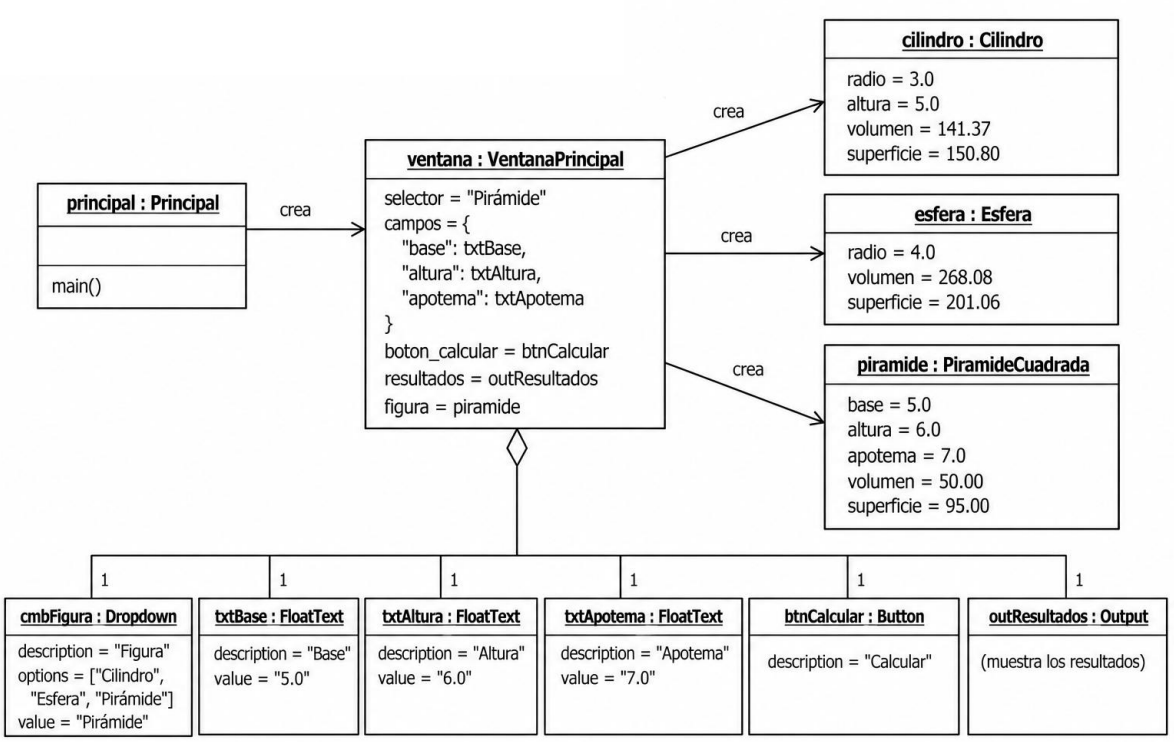
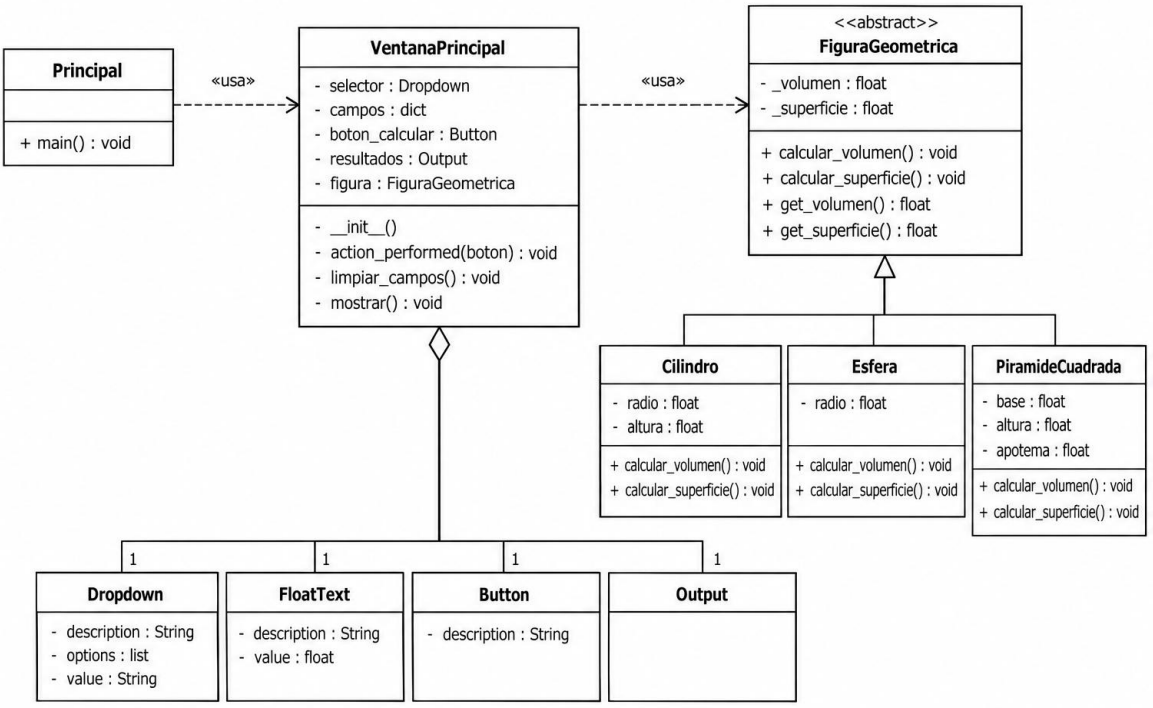
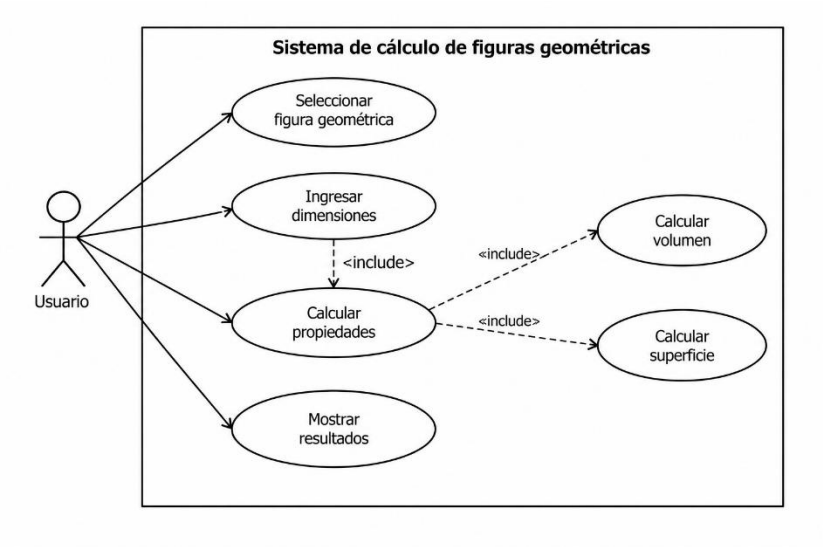



Diagrama de Clases



Casos de uso




Enlace de Github: https://github.com/mabarreraa-eng/Actividad-III-Poo.git